

AI WEATHERWISE

Project Development Manual

Deliver intelligent weather forecasts and insights

Role	Name
Leader	SRIRAMASAMY T
Member	DHINISHKUMAR P
Member	PRAVEENKUMAR R
Member	ILLAYARAYA A

Prepared as a practical record of project setup, development, database configuration, API testing and execution.

1. Project Overview

AI WeatherWise is a RESTful backend platform developed with Node.js and Express.js. The project uses MongoDB with Mongoose for data persistence and integrates Google Gemini AI to generate intelligent weather summaries and personalized recommendations from live environmental conditions. The platform also includes JWT authentication, bcrypt password hashing, favorite-location management, centralized error handling and a fallback mode for continued API operation when external API keys are unavailable.

This manual explains how the project was planned, configured, developed and tested, using the structure and terminology of the supplied project document.

1.1 Project Objective

- Provide real-time weather information through REST APIs.
- Convert raw weather metrics into useful natural-language summaries.
- Generate personalized activity and clothing recommendations using Gemini AI.
- Allow users to save and manage favorite cities securely.
- Protect secured routes with JWT authentication and role-based checks.

1.2 Main Users

User Type	Main Usage
Registered User	Manage favorite cities, fetch weather and request AI insights.
Developer / Admin	Monitor API health, user registrations and service integrations.
Public Weather User	Consume current weather endpoints without an account.

2. Software and Hardware Requirements

2.1 Software Requirements

Software	Purpose
Windows 10/11, macOS or Linux	Operating environment.
Node.js v16+	Runs the backend application.
npm v8+	Installs and manages dependencies.
MongoDB	Stores users and favorite locations.
Gemini AI SDK	Connects the application to Gemini AI.
Postman	Tests API requests and responses.
Visual Studio Code	Project development and editing.

2.2 Hardware Requirements

Component	Requirement
Processor	Intel Core i5 (8th Gen+) / AMD Ryzen 5 or equivalent
RAM	8 GB minimum; 16 GB recommended
Storage	1 GB available project workspace

2.3 Why These Requirements Are Needed

The backend depends on a Node.js runtime and local development tools. MongoDB must be available for database operations, while Postman or Thunder Client is used for API verification. The project is designed to run cross-platform.

3. Project Architecture

The application follows an MVC-oriented architecture that separates data models, request processing and API routing. The Controller layer coordinates weather fetching and AI insight generation, while the Model layer manages User and Location entities. In this headless backend, the routing layer serves as the interface for external clients.

3.1 Main Components

Component	Responsibility
Interface / API Client	Sends HTTP requests and receives JSON responses.
Express Server Gateway	Handles ports, HTTP methods, JSON parsing and CORS.
Authentication & Role Middleware	Validates tokens and checks route authorization.
Granular Services	Applies domain-specific business logic.
Database Interface	Uses Mongoose to communicate with MongoDB.
AI Service Integration	Generates weather summaries and recommendations.

3.2 MVC Flow

1. Client sends a request to an API route.
2. Express receives and validates the request.
3. Middleware checks authentication and authorization where required.
4. Controller processes the request and invokes the required model or service.
5. Mongoose performs database operations when required.
6. Gemini AI is called for weather summaries or recommendations when requested.
7. The controller returns a standard JSON response.

4. Project Setup - Step by Step

4.1 Create the Project Folder

1. Create a new folder named AI Weatherwise.
2. Open the folder in Visual Studio Code.
3. Open the VS Code terminal.

4.2 Initialize Node.js

Initialize the project with npm:

```
npm init -y
```

4.3 Install Required Dependencies

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer  
@google/genai
```

4.4 Install Nodemon

```
npm install --save-dev nodemon
```

4.5 Create the Initial Files

- index.js
- .env
- package.json
- .gitignore

5. Environment Configuration

The project stores runtime configuration in a `.env` file at the project root. This prevents configuration values from being hard-coded into source files.

5.1 Required Variables

```
PORT=5000  
  
MONGO_URI=your_mongodb_connection_string  
  
JWT_SECRET=your_jwt_secret  
  
GEMINI_API_KEY=your_gemini_api_key
```

5.2 Configuration Steps

1. Create a file named `.env` in the project root.
2. Add the `PORT` value for the Express server.
3. Add the MongoDB connection string in `MONGO_URI`.
4. Add a secret value for JWT signing in `JWT_SECRET`.
5. Add the Gemini API key in `GEMINI_API_KEY`.

Keep the `.env` file out of public source control. The supplied project document identifies these variables as the runtime configuration used by the backend.

6. Backend Development Manual

The backend is organized into modular layers for routing, controllers, middleware, models and utility modules. This structure is intended to improve maintainability and scalability.

6.1 Application Entry Point - index.js

The index.js file is the main entry point. It loads environment variables, configures middleware, connects to MongoDB, registers API routes and starts the server after a successful database connection.

6.2 Development Sequence

1. Load environment variables from .env.
2. Create the Express application.
3. Register JSON parsing, CORS and required middleware.
4. Establish the MongoDB connection through Mongoose.
5. Register authentication, weather, location and other API routes.
6. Start the server after the database is ready.

6.3 Authentication and Security

- JWT tokens are used to validate authenticated users.
- bcrypt is used for password hashing.
- Role-based route guarding protects secured operations.
- Request sanitization is used to reduce malicious parameter injection risks.

7. Database Configuration Manual

MongoDB is used for persistent storage and Mongoose provides the schema and database interface layer.

7.1 Install Mongoose

```
npm install mongoose
```

7.2 Database Setup Sequence

1. Start MongoDB and ensure the database service is available.
2. Set the MONGO_URI value in .env.
3. Create the required Mongoose schemas and models.
4. Establish the database connection before processing backend requests.

7.3 Main Data Entities

Entity	Important Fields
User	_id, name, email, password, role
Location	_id, user, city, country, createdAt

7.4 Relationship

A registered User can save multiple favorite Location records. The Location entity keeps a reference to its owning User, producing a one-to-many relationship.

8. Weather and AI Feature Development

8.1 Weather Data Flow

1. Receive the requested city/location from the client.
2. Validate the request and authenticated user context when required.
3. Fetch current weather information from the configured weather service.
4. Return the weather metrics through the REST API.

8.2 AI Insight Flow

1. Collect the current weather conditions and relevant temperature data.
2. Send the required information to the Gemini integration.
3. Generate a natural-language weather summary or recommendation.
4. Return the generated insight in the API response.

8.3 Personalized Recommendations

The project is designed to provide context-aware suggestions such as outdoor activity or clothing recommendations based on the live weather condition rather than returning only raw measurements.

8.4 Favorite Locations

- Create a favorite location for a user.
- Read saved locations.
- Update or remove location records when needed.

9. API Testing Manual

The supplied project document identifies Postman or Thunder Client as the tools used to verify the REST APIs and confirms API testing for authentication and weather-related endpoints.

9.1 Register API Test

1. Open Postman or Thunder Client.
2. Select the Register API endpoint.
3. Send the required user fields in JSON format.
4. Verify that the server returns the expected registration response.

9.2 Login API Test

1. Send valid registered credentials to the Login API.
2. Verify that authentication succeeds.
3. Store the returned JWT for testing protected routes.

9.3 Weather API Test

1. Call the Weather API with the requested location.
2. Verify current weather fields in the JSON response.

9.4 Recommendation API Test

1. Send the relevant weather/location input.
2. Verify that the response contains an AI-generated recommendation or summary.

10. Running the Project

10.1 Install Dependencies

```
npm install
```

10.2 Start the Development Server

```
npm start
```

10.3 Successful Execution

After startup, the backend connects to MongoDB and begins listening for incoming API requests. Before execution, MongoDB must be running and all required environment variables must be configured.

10.4 Execution Checklist

- MongoDB service is running
- MONGO_URI is configured
- JWT_SECRET is configured
- GEMINI_API_KEY is configured
- Node.js dependencies are installed
- API client is ready for testing

11. Complete User Flow

1. User registers or logs in.
2. User saves a favorite city through the API.
3. The backend stores the location in MongoDB.
4. User requests current weather for the city.
5. The backend retrieves the weather data.
6. User requests an AI-generated insight.
7. The backend sends relevant weather conditions to Gemini.
8. The server returns the summary or recommendation as JSON.
9. JWT validation and route permissions are enforced on protected operations.

11.1 Error Handling and Fallback

The project includes centralized error handling and a fallback mode so the API can continue operating in a reduced-capability state when external API keys are unavailable. This is intended to improve resilience during development and testing.

12. Team Roles and Responsibilities

Role	Team Member
Leader	SRIRAMASAMY T
Member	DHINISHKUMAR P
Member	PRAVEENKUMAR R
Member	ILLAYARAYA A

12.1 Suggested Project Work Breakdown

The following is a practical documentation-oriented breakdown for the four-person team. It is presented as a project organization aid; the supplied source does not assign individual tasks to specific members.

Work Area	Activities
Project Coordination	Planning, integration and final review
Backend Development	Express routes, controllers, middleware and services
Database	MongoDB connection, Mongoose schemas and persistence
Testing & Documentation	Postman testing, execution checks, screenshots and manual preparation

13. Project Completion Summary

The AI WeatherWise project combines a Node.js/Express REST backend, MongoDB persistence, secure authentication and Gemini-based AI insights. The development process begins with environment and dependency setup, continues through application architecture and database configuration, and concludes with API testing and local execution.

13.1 Final Verification

- Project folder created and initialized
- Required dependencies installed
- Environment variables configured
- MongoDB connection established
- Backend routes available
- Authentication tested
- Weather API tested
- AI recommendation API tested
- Local server started successfully

13.2 Team

Leader: SRIRAMASAMY T

Members: DHINISHKUMAR P, PRAVEENKUMAR R, ILLAYARAYA A

This manual is based on the supplied AI WeatherWise project document, including its architecture, setup, configuration, database, testing and execution sections.